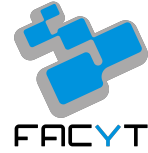




Universidad de Carabobo
Facultad Experimental de Ciencias y Tecnología
Arquitectura del Computador
Sección 1



Informe

Práctica de Laboratorio 4

Integrantes:

Penélope Robledo, C.I. 32.482.938

Nathalia Noguera, C.I. 32.756.046

Profesor:

José Canache

Naguanagua, agosto de 2026

1. Explique con un diagrama cómo funciona el ciclo de una interrupción de hardware (desde que ocurre el evento hasta que se atiende).

Ciclo completo de una interrupción de hardware (MIPS32)

Desde que ocurre el evento hasta que se retorna al programa original

1. Generación del evento externo

- Un dispositivo periférico (como un teclado o un temporizador) genera una señal física de interrupción al requerir atención inmediata del procesador.



2. Detección y verificación

- El procesador comprueba de manera continua (al finalizar cada instrucción) si existe una señal pendiente y si las interrupciones están habilitadas en su registro de estado.



3. Acciones automáticas del hardware

- El procesador suspende de forma automática el flujo actual del programa.
- Guarda la dirección de retorno en el registro EPC (Exception Program Counter).
- Deshabilita temporalmente las interrupciones para evitar anidamientos no controlados.
- Salta de manera forzosa hacia la dirección fija del vector de interrupción o manejador de excepciones.



4. Atención por software (Rutina de Servicio - ISR)

- La rutina de servicio respalda el contexto de los registros generales en la pila para evitar corromper los datos del programa original.
- Consulta los registros del sistema para identificar la fuente exacta de la interrupción.
- Ejecuta la tarea requerida (por ejemplo, leer el búfer del teclado o reiniciar el temporizador) y limpia la señal en el dispositivo.
- Restaura los registros que habían sido guardados al inicio de la rutina.



5. Retorno y continuación

- Se ejecuta la instrucción de retorno de excepción (`eret` en MIPS32).
- El procesador utiliza el valor almacenado en el EPC para devolver el control exactamente al punto donde fue interrumpido el programa original, reanudando su ejecución de forma transparente.

Leyenda de colores: ■ Evento externo ■ Detección ■ Acciones de hardware ■ Rutina de servicio ■ Retorno y continuación

2.¿Qué diferencias hay entre gestionar E/S con sondeo y hacerlo con interrupciones?

La gestión de la entrada/salida (E/S) mediante sondeo, también conocido como polling, consiste en que el procesador revisa de forma continua y periódica el estado de los dispositivos periféricos para determinar si están listos para realizar una operación de lectura o escritura. En este enfoque, el programa principal incluye bucles explícitos en los que se consulta repetidamente un registro de estado del dispositivo (por ejemplo, si hay un carácter disponible en el teclado o si ha finalizado una temporización). Mientras se realiza esta consulta constante, el procesador permanece ocupado en esa tarea y no puede dedicar ciclos a otras operaciones útiles, lo que genera un consumo innecesario de recursos cuando el dispositivo no tiene datos pendientes.

Por el contrario, la gestión de E/S mediante interrupciones permite que el procesador continúe ejecutando el programa principal sin necesidad de comprobar continuamente el estado de los dispositivos. Cuando un periférico necesita atención (por ejemplo, se pulsa una tecla o se agota un temporizador), genera una señal de interrupción que detiene temporalmente la ejecución normal. El procesador guarda el contexto actual, salta a una rutina de servicio de interrupción específica, atiende la petición del dispositivo y, una vez finalizada, restaura el contexto y reanuda el programa principal exactamente donde lo había dejado. De este modo, el procesador solo interviene cuando realmente es necesario.

Una de las diferencias más relevantes se encuentra en la eficiencia del uso del procesador:

- Con sondeo: el tiempo de CPU se desperdicia en comprobaciones inútiles durante los periodos de inactividad del dispositivo, lo que reduce el rendimiento global del sistema, especialmente si se deben atender varios periféricos o si las operaciones de E/S son esporádicas.
- Las interrupciones liberan al procesador para que pueda realizar otras tareas o incluso entrar en modos de bajo consumo mientras espera eventos, mejorando significativamente la eficiencia y permitiendo sistemas más responsivos.

Otra diferencia importante radica en la complejidad del software y el control temporal, ya

que el sondeo resulta conceptualmente más sencillo de implementar, ya que todo el flujo de control permanece dentro del programa principal y no requiere la configuración de vectores de interrupción, prioridades ni salvaguarda de registros. Sin embargo, este método dificulta el cumplimiento de requisitos de tiempo real precisos, porque el intervalo entre comprobaciones depende de la duración del bucle de sondeo. Las interrupciones, aunque exigen un diseño más cuidadoso (manejo de la pila, reentrada, posibles anidamientos y sincronización con el código principal), ofrecen tiempos de respuesta mucho más predecibles y cortos, ya que el dispositivo avisa de inmediato cuando está listo.

Desde el punto de vista de la arquitectura del sistema, el sondeo no necesita hardware adicional de notificación, mientras que las interrupciones requieren que el procesador y los periféricos soporten líneas de petición de interrupción, un controlador de interrupciones y mecanismos de priorización. En el contexto de programas como los propuestos (buffer circular de teclado o semáforo controlado por tecla y temporizadores), el uso de interrupciones permite que el sistema reaccione de forma asíncrona a la llegada de caracteres o al vencimiento de temporizadores sin bloquearse en bucles de espera activos, facilitando la implementación de comportamientos concurrentes aparentes en un único procesador.

Aunque ambas técnicas permiten gestionar la comunicación con dispositivos externos, el sondeo prioriza la simplicidad a costa de un uso ineficiente del procesador, mientras que las interrupciones privilegian la eficiencia y la reactividad a costa de una mayor complejidad de programación y de requisitos hardware más elaborados. La elección entre una u otra depende de las exigencias de rendimiento, los plazos temporales y los recursos disponibles del sistema.

3. ¿Qué ventajas tiene el uso de interrupciones en términos de uso del procesador?

El uso de interrupciones aporta importantes ventajas en lo que respecta al aprovechamiento del procesador, especialmente cuando se comparan con técnicas de gestión de entrada/salida basadas en sondeo continuo.

La principal ventaja radica en que el procesador deja de estar permanentemente ocupado comprobando el estado de los dispositivos periféricos. En lugar de ejecutar de forma reiterada bucles de consulta, el procesador puede dedicarse a otras tareas del programa principal o, en su defecto, permanecer en un estado de espera pasiva hasta que realmente se produzca un evento relevante. De este modo, los ciclos de reloj se destinan a trabajo útil en lugar de a comprobaciones inútiles, lo que se traduce en un uso mucho más eficiente de la capacidad de cómputo disponible.

Esta eficiencia se hace especialmente visible en aplicaciones de los ejercicios propuestos, donde es necesario atender simultáneamente la entrada de caracteres por teclado y el control de temporizadores de distinta duración. Con interrupciones, el procesador puede continuar ejecutando el código principal o incluso permanecer en un bucle de espera de baja carga mientras el teclado o el temporizador no generan actividad.

Solo cuando se produce una pulsación válida o se agota un intervalo de tiempo, el dispositivo correspondiente genera una señal de interrupción que desvía temporalmente el flujo de ejecución hacia la rutina de servicio adecuada. Una vez atendida la petición, el procesador recupera el contexto anterior y reanuda su actividad normal, sin haber desperdiciado tiempo en sondeos constantes.

Otra ventaja significativa es la mejora en la capacidad de respuesta y en la multiplexación temporal de tareas, debido a que al no estar bloqueado en un bucle de sondeo, el procesador puede atender de forma casi inmediata los eventos asíncronos, lo que permite implementar comportamientos concurrentes aparentes en un único hilo de ejecución. En el caso del buffer circular de teclado, por ejemplo, las interrupciones de teclado permiten almacenar únicamente las letras mayúsculas en el momento exacto en que llegan, mientras el temporizador de 20

segundos avanza de forma independiente.

De manera análoga, en la simulación del semáforo, las interrupciones de teclado y de temporizador posibilitan que el sistema reaccione a la tecla “s” y gestione los cambios de estado (verde, amarillo y rojo) sin necesidad de que el procesador se mantenga continuamente comprobando el paso del tiempo o el estado del pulsador.

Además, el uso de interrupciones reduce el consumo de recursos del sistema en términos de ciclos de procesador y, en arquitecturas que lo permiten, puede facilitar modos de bajo consumo energético, al evitar la actividad continua de sondeo, se disminuye la carga térmica y se optimiza el rendimiento global, especialmente cuando los eventos de entrada/salida son esporádicos o cuando se deben gestionar varios dispositivos al mismo tiempo. Esta característica resulta particularmente valiosa en sistemas embebidos o en programas que deben mantener un comportamiento predecible durante largos periodos, como los intervalos de 10, 20 o 30 segundos contemplados en el ejercicio del semáforo.

En conjunto, las interrupciones permiten que el procesador actúe de manera orientada a eventos en lugar de de forma proactiva y continua. El resultado es un aprovechamiento más racional de su capacidad de cálculo, debido a que logra:

- Maximizar el tiempo dedicado a operaciones productivas
- Minimizar el tiempo perdido en esperas activas
- Facilitar la implementación de aplicaciones reactivas y temporales complejas.

Estas ventajas explican por qué, en los ejercicios propuestos, la gestión mediante interrupciones resulta considerablemente más adecuada que el sondeo para lograr un comportamiento eficiente y correctamente sincronizado.

4. ¿Qué registros especiales se utilizan en MIPS32 para gestionar interrupciones?

En la arquitectura MIPS32, la gestión de interrupciones se realiza principalmente a través de un conjunto de registros especiales pertenecientes al Coprocesador 0 (CP0). Estos registros no forman parte del banco de registros de propósito general y solo pueden ser leídos o escritos mediante instrucciones específicas del sistema, como `mfc0` (move from coprocessor 0) y `mtc0` (move to coprocessor 0).

Su función fundamental es permitir que el procesador detecte, clasifique y retorne de manera controlada de cualquier evento asíncrono o excepción, tanto de hardware como de software.

El registro Status (registro 12 del CP0) constituye el principal elemento de control del sistema; en él se encuentran los bits que habilitan o deshabilitan globalmente las interrupciones (como el bit IE), así como las máscaras individuales (IM) que permiten seleccionar qué fuentes de interrupción están autorizadas para interrumpir al procesador en un momento determinado.

Asimismo, contiene indicadores críticos como el nivel de excepción (EXL) y el modo de operación del procesador. Cuando se produce una interrupción, el hardware modifica automáticamente ciertos bits de este registro para impedir que nuevas interrupciones se aniden de forma no controlada, garantizando así la integridad del contexto de ejecución.

El registro Cause (registro 13 del CP0) actúa como el mecanismo principal de diagnóstico, ya que su contenido informa al software de la causa exacta que provocó la desviación del flujo normal de ejecución. Dentro de este registro se almacenan tanto el código de excepción como los bits de interrupciones pendientes (IP), los cuales reflejan qué dispositivos externos o temporizadores han solicitado atención.

Gracias a esta información, la Rutina de Servicio de Interrupción (ISR) puede determinar rápidamente si el evento proviene del teclado, de un temporizador o de otra fuente, y actuar en consecuencia sin necesidad de realizar sondeos (*polling*) innecesarios.

El Exception Program Counter, conocido como EPC (registro 14 del CP0), almacena la dirección de la instrucción que estaba ejecutándose o a punto de ejecutarse en el instante exacto en

que se produjo la interrupción. Este valor resulta imprescindible para poder retornar al punto exacto del programa principal una vez finalizada la atención del evento.

Tras completar las acciones correspondientes en la rutina de servicio, el procesador utiliza la instrucción especial *eret* (*exception return*), la cual restaura el flujo utilizando el valor de EPC en el contador de programa y recupera el estado previo resguardado en el registro Status.

Además de los tres registros anteriores, en escenarios que involucran control temporal (como los intervalos de 10, 20 o 30 segundos requeridos en el ejercicio del semáforo, o la ventana de 20 segundos del buffer de teclado) adquieren especial relevancia los registros Count y Compare del Coprocesador 0. El registro Count (registro 9) se incrementa de forma continua a una frecuencia determinada por el reloj del sistema, mientras que el registro Compare (registro 11) permite programar el umbral temporal en el que se generará una interrupción de temporizador.

Cuando el valor de Count alcanza al de Compare, se activa automáticamente el bit correspondiente de interrupción pendiente en el registro Cause, lo que facilita la implementación precisa de esperas temporales y cambios de estado sin depender de bucles de retardo activos.

En conjunto, la interacción coordinada de estos registros especializados permite que el procesador responda de manera ordenada y eficiente a los eventos externos. Al producirse una interrupción, el hardware actualiza Cause y EPC, modifica Status para bloquear interrupciones adicionales y transfiere el control a la dirección fija del manejador de excepciones. Posteriormente, el software lee Cause para identificar el origen del evento, ejecuta las acciones necesarias como almacenar un carácter en el búfer circular o actualizar el estado del semáforo, y finalmente, utiliza EPC y Status para reanudar la ejecución normal.

5. ¿Por qué es necesario guardar el contexto (registros) al entrar en una rutina de servicio?

La necesidad de guardar el contexto, es decir, el contenido de los registros del procesador, al entrar en una rutina de servicio de interrupción responde a un principio fundamental de la correcta ejecución de programas en cualquier arquitectura, incluida MIPS32. Cuando se produce una interrupción, el procesador detiene de manera abrupta la instrucción que estaba ejecutando o a punto de ejecutar y transfiere el control a la rutina de servicio. En ese instante, los registros de propósito general contienen valores intermedios, direcciones, contadores o datos que el programa principal estaba utilizando, si la rutina de servicio modifica alguno de esos registros sin haberlos preservado previamente, al regresar al punto de interrupción el programa principal continuará su ejecución con valores alterados, lo que provoca resultados incorrectos, comportamientos impredecibles o incluso fallos graves del sistema.

El contexto no se limita únicamente a los registros de propósito general, aunque el hardware de MIPS32 se encarga automáticamente de salvar ciertos elementos críticos, como el valor del contador de programa en el registro EPC y algunos bits del registro Status, el resto del estado del procesador permanece bajo la responsabilidad del software.

La rutina de servicio suele necesitar utilizar registros para realizar operaciones: leer el carácter del teclado, actualizar el buffer circular, modificar variables de estado del semáforo o manipular contadores de tiempo, cada vez que la rutina escribe en un registro, sobrescribe el valor que tenía el programa interrumpido. Por tanto, antes de realizar cualquier modificación, es imprescindible copiar el contenido original de los registros que se van a utilizar (normalmente en la pila) y restaurarlo de forma exacta antes de ejecutar la instrucción de retorno de excepción.

Esta salvaguarda del contexto garantiza la transparencia de la interrupción desde el punto de vista del programa principal, de esta forma, el código que estaba ejecutándose no debe percibir de que ha sido interrumpido; al reanudarse, debe encontrar exactamente el mismo estado de registros que tenía en el momento de la interrupción.

Sin esta protección, cualquier cálculo intermedio, cualquier puntero o cualquier valor temporal se corrompería, rompiendo la lógica del algoritmo. Un ejemplo de esto serían los ejercicios

propuestos: el programa principal puede estar en medio de la gestión del buffer circular o esperando el siguiente cambio de estado del semáforo, si la rutina de servicio de teclado o de temporizador altera registros sin restaurarlos, el buffer podría perder caracteres, los temporizadores podrían desincronizarse o el mensaje mostrado en pantalla podría resultar incorrecto.

Además, la guarda del contexto adquiere aún mayor importancia cuando existe la posibilidad de interrupciones anidadas o de múltiples fuentes de interrupción activas. Si una interrupción de mayor prioridad llega mientras se está atendiendo otra, la nueva rutina de servicio también modificará registros.

Solo si cada nivel de interrupción preserva y restaura correctamente el contexto que encontró al entrar, se mantiene la integridad de todos los niveles de ejecución, aunque en muchos sistemas educativos se deshabilitan temporalmente las interrupciones dentro de la rutina de servicio para simplificar el diseño, el principio sigue siendo el mismo: el estado del procesador debe quedar intacto al regresar.

Desde el punto de vista de la implementación, la forma habitual de guardar el contexto consiste en utilizar la pila del sistema, al comenzar la rutina de servicio se reserva espacio en la pila y se almacenan los registros que se van a emplear; al finalizar, se recuperan en orden inverso y se libera el espacio.

Este procedimiento, aunque añade unas pocas instrucciones al comienzo y al final de la rutina, es el precio necesario para preservar la corrección del programa, ya que sin él, la potencia y la flexibilidad que ofrecen las interrupciones se convertirían en una fuente constante de errores difíciles de depurar.

De este modo, guardar el contexto al entrar en una rutina de servicio constituye una condición indispensable para que el mecanismo de interrupciones funcione de manera fiable, solo mediante esta preservación del estado del procesador se logra que el programa principal continúe su ejecución como si la interrupción nunca hubiera ocurrido, manteniendo la coherencia de los datos y la predictibilidad del comportamiento del sistema.

6. Momentos en que pueden generarse excepciones en un sistema MIPS32.

En un sistema basado en la arquitectura MIPS32, las excepciones constituyen eventos que interrumpen el flujo normal de ejecución de un programa para permitir que el procesador o el sistema operativo atiendan una condición anómala o especial. Estas situaciones pueden originarse tanto por causas internas al propio programa como por condiciones detectadas por el hardware.

a) Enumera al menos 4 situaciones en las que se pueda generar una excepción.

Una de las causas más clásicas es el desbordamiento aritmético o overflow, este evento se produce cuando una operación de suma o resta realizada con instrucciones que detectan desbordamiento (como add o sub) genera un resultado que no puede representarse correctamente dentro del rango de los 32 bits del registro destino, así, el hardware del procesador comprueba los bits de signo de los operandos y del resultado; si detecta una inconsistencia que indica desbordamiento, genera de inmediato una excepción.

Esta protección resulta fundamental en programas que realizan cálculos numéricos críticos, ya que evita que un valor erróneo se propague y corrompa posteriores operaciones.

Otra situación frecuente es el error de dirección o address error, que se genera cuando el procesador intenta acceder a memoria utilizando una dirección que no cumple las reglas de alineación exigidas por la arquitectura o que se encuentra fuera del espacio de direcciones válido.

En MIPS32, por ejemplo, una lectura o escritura de una palabra de 32 bits debe realizarse en una dirección múltiplo de cuatro; si la dirección no está alineada, el hardware detecta la irregularidad y provoca la excepción, del mismo modo, cualquier intento de acceder a una zona de memoria no mapeada o protegida también desencadena este tipo de excepción, protegiendo así la integridad del sistema y evitando accesos ilegales.

Una tercera causa relevante es la ejecución de una instrucción ilegal o reservada, que se pro-

duce cuando el procesador encuentra en el flujo de instrucciones un código de operación que no corresponde a ninguna instrucción válida de la arquitectura MIPS32, o que pertenece a un conjunto de instrucciones no implementado en ese procesador concreto, se genera una excepción de instrucción reservada.

Esta situación puede aparecer por errores de programación, por corrupción de memoria o por intentos de ejecutar código destinado a una versión posterior de la arquitectura, tal mecanismo de excepción permite al sistema detectar el problema y transferir el control a una rutina de tratamiento adecuada, en lugar de continuar con un comportamiento indefinido.

La cuarta situación significativa corresponde a la ejecución deliberada de instrucciones especiales de control, concretamente la instrucción `syscall` (llamada al sistema) y la instrucción `break` (punto de ruptura). La instrucción `syscall` se utiliza para solicitar servicios al sistema operativo o al entorno de ejecución; al encontrarla, el procesador genera una excepción de forma intencionada para transferir el control al manejador correspondiente, de manera análoga, la instrucción `break` provoca una excepción de breakpoint, empleada habitualmente por depuradores o por el propio programa para detener la ejecución en un punto concreto y examinar el estado del sistema.

Ambas constituyen excepciones programadas que permiten una comunicación controlada entre el código de usuario y los niveles privilegiados del sistema.

Estas cuatro situaciones (desbordamiento aritmético, error de dirección, instrucción ilegal o reservada, y ejecución de `syscall` o `break`) ilustran la diversidad de condiciones que pueden originar una excepción en MIPS32.

Cada una de ellas es detectada por mecanismos hardware específicos que interrumpen el flujo normal de instrucciones y preparan al procesador para que el software de tratamiento pueda identificar la causa y actuar en consecuencia, garantizando así la robustez y la seguridad de la ejecución.

b) Explica qué etapas del pipeline pueden provocar una excepción y por qué.

Una vez identificadas las principales situaciones que pueden originar una excepción en un sistema MIPS32, resulta necesario analizar en qué momentos concretos del ciclo de ejecución de una instrucción se detectan estas condiciones.

En la arquitectura MIPS32, las instrucciones se procesan a través de un pipeline de cinco etapas clásicas:

1. Captación de instrucción (IF)
2. Decodificación (ID)
3. Ejecución (EX)
4. Acceso a memoria (MEM)
5. Escritura de resultados (WB).

Cada una de estas etapas realiza comprobaciones específicas, de modo que una excepción puede generarse en distintas fases del pipeline según la naturaleza del problema detectado.

La etapa de captación de instrucción (IF) es la primera en la que puede producirse una excepción, en ella el procesador obtiene la siguiente instrucción a partir del valor del contador de programa. Si la dirección contenida en el PC no está alineada correctamente (por ejemplo, no es múltiplo de cuatro) o apunta a una zona de memoria inválida o no mapeada, el hardware genera una excepción de error de dirección de instrucción. Asimismo, cuando existe un sistema de memoria virtual, un fallo de TLB en la traducción de la dirección de la instrucción también se detecta en esta etapa.

Estas comprobaciones son imprescindibles porque garantizan que solo se introduzcan en el pipeline direcciones de instrucción válidas; de lo contrario, el procesador podría intentar decodificar datos erróneos o acceder a regiones protegidas.

En la etapa de decodificación (ID) se identifican otro conjunto importante de excepciones, durante esta fase se examina el código de operación de la instrucción captada. Si el código no

corresponde a ninguna instrucción válida de la arquitectura MIPS32, o si se trata de una instrucción reservada no implementada en el procesador concreto, se genera una excepción de instrucción ilegal o reservada. También en esta etapa se reconocen las instrucciones especiales `syscall` y `break`, que provocan excepciones de forma deliberada.

La detección temprana en ID permite evitar que una instrucción inválida avance innecesariamente por el resto del pipeline y consuma recursos o produzca efectos colaterales indeseados.

La etapa de ejecución (EX) es el lugar donde se originan las excepciones de naturaleza aritmética y algunas trampas programadas, en ella se realizan las operaciones de cálculo con la unidad aritmético-lógica. Cuando se ejecutan instrucciones como `add` o `sub` que incorporan detección de desbordamiento, el hardware comprueba si el resultado excede el rango representable en 32 bits; en caso afirmativo se genera la excepción de overflow, así como otras condiciones, como ciertos traps o, en implementaciones que lo contemplan, la división por cero, también se detectan durante el cálculo.

Esta etapa resulta crítica porque es el momento en que los operandos ya están disponibles y la operación se materializa, permitiendo una comprobación precisa de la validez del resultado antes de que este se escriba en un registro o se utilice en etapas posteriores.

La etapa de acceso a memoria (MEM) constituye otro punto fundamental de generación de excepciones, en este módulo se efectúan las lecturas y escrituras de datos solicitadas por las instrucciones de carga y almacenamiento. Si la dirección de datos no está correctamente alineada, o si apunta a una región de memoria inválida, protegida o no traducida por el TLB, se produce una excepción de error de dirección de datos o de fallo de página.

Estas comprobaciones protegen la integridad del sistema de memoria y evitan que se realicen accesos ilegales que podrían corromper datos o vulnerar mecanismos de protección, ya que al producirse en MEM, la excepción detiene la instrucción antes de que el valor erróneo se escriba en un registro o se almacene en memoria.

Finalmente, la etapa de escritura de resultados (WB) no suele generar excepciones propias, su función se limita a actualizar los registros de propósito general con los resultados ya calculados y validados en etapas anteriores. Por esta razón, las condiciones anómalas se detectan y tratan antes de llegar a WB, lo que facilita el mantenimiento de excepciones precisas: el pro-

cesador puede identificar exactamente qué instrucción causó el problema y dejar el estado del sistema de forma coherente para el manejador de excepciones.

7. Estrategias de tratamiento de excepciones e interrupciones.

En la arquitectura MIPS32, los términos excepción e interrupción se utilizan con frecuencia de manera relacionada, aunque presentan diferencias conceptuales y de origen que resultan fundamentales para comprender el tratamiento de eventos anómalos o externos. De forma general, una excepción se entiende como cualquier evento que provoca una desviación del flujo normal de ejecución de un programa y transfiere el control a una rutina de tratamiento especializada. Dentro de este concepto amplio, las interrupciones constituyen un subconjunto particular de excepciones, caracterizado por su origen y por su relación temporal con las instrucciones que se están ejecutando.

a) Explica las diferencias entre interrupciones y excepciones (¿son síncronas o asíncronas?).

La principal diferencia radica en el carácter síncrono o asíncrono del evento. Las excepciones son, en su mayoría, síncronas, esto significa que se generan como consecuencia directa de la ejecución de una instrucción concreta y en un momento preciso del ciclo de esa instrucción. Ejemplos típicos son el desbordamiento aritmético detectado durante una operación de suma o resta, el intento de ejecutar una instrucción ilegal o reservada, la aparición de una instrucción syscall o break, o un error de dirección producido al intentar acceder a una posición de memoria no alineada o inválida.

En todos estos casos, el evento está ligado a una instrucción determinada: si se volviera a ejecutar el mismo programa con los mismos datos, la excepción se produciría exactamente en el mismo punto. Esta propiedad de sincronía facilita el tratamiento preciso, ya que el procesador puede identificar con exactitud qué instrucción provocó el problema y dejar el estado del sistema de forma coherente para el manejador.

Por el contrario, las interrupciones son eventos asíncronos, ya que su aparición no depende de la instrucción que el procesador está ejecutando en ese instante, sino de circunstancias externas o independientes del flujo de instrucciones, teniendo como fuentes más habituales los dispositivos de entrada/salida (como el teclado), los temporizadores o señales externas de

hardware.

Una interrupción puede ocurrir en cualquier momento, incluso entre dos instrucciones consecutivas o durante la ejecución de una instrucción de larga duración, por esta razón, no existe una relación directa y predecible entre la instrucción en curso y el evento que genera la interrupción, así, el procesador debe estar preparado para atender estas señales en puntos seguros del pipeline, sin que ello implique necesariamente un error en el programa que se estaba ejecutando.

Otra diferencia relevante se encuentra en la intencionalidad y en la naturaleza del evento. Las excepciones síncronas suelen indicar una condición excepcional o anómala relacionada con el propio programa: un error de programación, una operación no permitida o una solicitud deliberada de servicio al sistema (como ocurre con `syscall`). Las interrupciones, en cambio, representan solicitudes de atención procedentes del entorno externo, no constituyen necesariamente un error; son mecanismos normales de comunicación entre los periféricos y el procesador. El programa principal puede estar funcionando de manera completamente correcta cuando llega una interrupción, y tras atenderla debe poder reanudarse como si nada hubiera ocurrido.

Desde el punto de vista del hardware, ambas categorías se gestionan a través de los mismos mecanismos generales de la arquitectura MIPS32 (registros del Coprocesador 0, vector de excepciones, etc.). Sin embargo, el registro `Cause` permite distinguir el origen concreto del evento, de modo que la rutina de tratamiento puede determinar si se trata de una excepción síncrona asociada a una instrucción o de una interrupción asíncrona generada por una fuente externa.

Esta distinción resulta esencial para aplicar la estrategia de recuperación adecuada: en el caso de una excepción síncrona puede ser necesario corregir el error, terminar el proceso o simular el efecto de la instrucción; en el caso de una interrupción asíncrona, normalmente basta con atender al dispositivo y devolver el control al programa interrumpido sin alterar su lógica. En resumen, aunque en MIPS32 tanto las excepciones como las interrupciones se canalizan por un camino común de tratamiento, se diferencian claramente por su carácter temporal y por su origen. Las excepciones son predominantemente síncronas y están ligadas a la ejecución de instrucciones concretas, mientras que las interrupciones son asíncronas y responden a eventos externos independientes del flujo de instrucciones. Esta distinción permite al sistema ofrecer

respuestas precisas y eficientes ante la amplia variedad de situaciones que pueden interrumpir la ejecución normal de un programa.

b) Describe brevemente dos estrategias para tratar excepciones en un sistema MIPS32 ¿Cómo se redirige la ejecución hacia la rutina de servicio? ¿Cuál es la función del registro EPC (Exception Program Counter)?

En un sistema MIPS32 existen diversas estrategias para tratar las excepciones e interrupciones, todas ellas apoyadas en el mecanismo hardware del Coprocesador 0 y en el software del manejador. Dos de las estrategias más representativas y ampliamente utilizadas se describen a continuación.

La primera estrategia consiste en el empleo de un vector de excepción común o punto de entrada único, en este enfoque, cualquier excepción o interrupción, independientemente de su origen, provoca que el hardware redirija automáticamente la ejecución hacia una dirección fija predeterminada, normalmente la dirección 0x80000180 en el espacio de kernel.

Una vez que el control se encuentra en esa dirección, la rutina de servicio examina el contenido del registro Cause para identificar la causa concreta del evento (código de excepción o bits de interrupciones pendientes) y, a partir de esa información, decide qué acciones realizar.

Esta estrategia simplifica el diseño del hardware, ya que solo se necesita un punto de entrada, y delega en el software la tarea de discriminar y tratar cada tipo de evento de manera adecuada, de igual forma, es la estrategia que se observa con mayor frecuencia en entornos educativos y en implementaciones clásicas de MIPS32. La segunda estrategia se basa en la utilización de vectores de excepción diferenciados o en el despacho software más elaborado mediante tablas de salto. En este caso, el sistema puede configurar distintos puntos de entrada según el tipo de evento (por ejemplo, un vector específico para interrupciones de temporizador, otro para excepciones de TLB o otro para errores de dirección). Aunque el hardware de MIPS32 base suele ofrecer un número limitado de vectores, el software puede emular un comportamiento vectorizado construyendo una tabla de direcciones indexada por el código de excepción contenido en el registro Cause.

De este modo, tras llegar al punto de entrada general, se calcula la dirección de la rutina específica y se salta directamente a ella. Esta estrategia permite un tratamiento más modular y eficiente cuando existen muchas fuentes distintas de excepciones, ya que evita largas cadenas de comprobaciones condicionales dentro de un único manejador.

En ambas estrategias la redirección de la ejecución hacia la rutina de servicio se realiza de forma automática por el hardware en el momento en que se detecta el evento. El procesador completa o abandona de manera controlada la instrucción en curso, actualiza los registros del Coprocesador 0 (entre ellos Status y Cause), almacena en el registro EPC la dirección de retorno apropiada y carga en el contador de programa la dirección del vector de excepción correspondiente.

A partir de ese instante, el flujo de instrucciones pasa a ejecutar el código del manejador situado en esa dirección. Todo este proceso ocurre sin que el programa interrumpido tenga que realizar ninguna acción explícita; es el propio hardware quien se encarga de transferir el control de manera segura y ordenada.

El registro EPC (Exception Program Counter) desempeña un papel esencial en este mecanismo, su función principal es almacenar la dirección de memoria de la instrucción que debe reanudarse una vez finalizado el tratamiento de la excepción o interrupción.

En el caso de las interrupciones asíncronas, EPC suele contener la dirección de la instrucción que estaba a punto de ejecutarse cuando llegó el evento. En el caso de las excepciones síncronas, puede contener la dirección de la propia instrucción que provocó el problema o la de la instrucción siguiente, según el tipo de excepción. Gracias a este valor, la instrucción `eret` (Exception Return) puede restaurar el contador de programa de forma precisa, permitiendo que el programa original continúe su ejecución exactamente en el punto correcto y con el estado de registros previamente salvado.

Sin el registro EPC, el procesador no tendría forma fiable de saber a qué dirección debe regresar tras atender el evento, lo que haría imposible un tratamiento transparente y correcto de las excepciones e interrupciones.

8. Habilitación de interrupciones en dispositivos y procesador.

La habilitación de interrupciones constituye un paso previo indispensable para que un sistema MIPS32 pueda reaccionar de forma asíncrona a los eventos generados por los dispositivos de entrada/salida. Sin esta configuración, ni el teclado ni la pantalla ni el propio procesador estarán preparados para generar o aceptar señales de interrupción, de modo que el sistema quedaría limitado a técnicas de sondeo continuo.

A continuación se explica de manera detallada cómo se lleva a cabo esta habilitación en cada uno de los elementos implicados.

a) Explica cómo se habilitan las interrupciones en:

El teclado:

En el teclado, la habilitación de interrupciones se realiza a través de su registro de control, situado en la dirección de memoria 0xffff0000. El bit 1 de este registro es el bit de habilitación de interrupción, para activarlo, el programa escribe el valor 2 en dicha dirección mediante una instrucción de almacenamiento (sw).

Una vez realizado este paso, cada vez que el usuario pulsa una tecla y el carácter queda disponible en el registro de datos del teclado (0xffff0004), el dispositivo genera automáticamente una señal de interrupción dirigida al procesador. Si este bit permanece en cero, el teclado sigue recibiendo caracteres, pero no avisa al procesador.

La pantalla:

En la pantalla (transmisor de la consola), el procedimiento es idéntico, aunque se utiliza el registro de control ubicado en la dirección 0xffff0008. El bit 1 de este registro controla la generación de interrupciones de la pantalla. Cuando se escribe el valor 2 en esa dirección, la pantalla genera una interrupción cada vez que está lista para aceptar un nuevo carácter.

Si el bit permanece en cero, la pantalla funciona únicamente en modo de sondeo y el programa debe consultar de forma continua el bit de “listo para transmitir”.

El procesador:

En el procesador, la habilitación de interrupciones se gestiona mediante el registro Status del Coprocesador 0 (registro número 12). Deben modificarse dos bits concretos:

- El bit 0, denominado IE (Interrupt Enable), actúa como interruptor global: si está a cero, el procesador ignora todas las señales de interrupción.
- El bit 8, correspondiente a la máscara de interrupción IM0, habilita específicamente el nivel de interrupción utilizado por el teclado y la pantalla en MARS.

La forma correcta de realizar esta configuración consiste en leer el valor actual del registro Status con la instrucción `mfc0`, activar los bits 0 y 8 mediante una operación OR con el valor `0x0101`, y escribir el resultado de nuevo en Status con la instrucción `mtc0`, solo cuando ambos bits están activos, el procesador acepta y atiende las interrupciones procedentes de los dispositivos.

De este modo, la habilitación completa de las interrupciones exige la configuración coordinada de tres elementos:

- El bit 1 del registro de control del teclado (`0xffff0000`)
- El bit 1 del registro de control de la pantalla (`0xffff0008`)
- Los bits 0 (IE) y 8 (IM0) del registro Status del procesador.

Únicamente cuando estos bits están correctamente activados, el sistema puede responder de manera eficiente y asíncrona a los eventos de entrada y salida.

b) ¿Qué pasaría si habilitamos interrupciones en los dispositivos, pero no en el procesador?

Una configuración incompleta de las interrupciones permite comprender con claridad la separación de responsabilidades que existe entre los dispositivos y el procesador en la arquitectura MIPS32. Cuando se habilitan las interrupciones únicamente en los periféricos y se dejan

deshabilitadas en el procesador, el sistema presenta un comportamiento muy concreto que conviene analizar en detalle.

Al escribir el valor 2 en el registro de control del teclado (0xffff0000) y en el registro de control de la pantalla (0xffff0008), ambos dispositivos quedan autorizados a generar señales de interrupción. En el momento en que el usuario pulsa una tecla, el teclado activa su línea de petición de interrupción. De igual forma, cuando la pantalla termina de procesar un carácter y está lista para recibir el siguiente, también genera su propia señal. Estas señales llegan al procesador y provocan que se activen los bits de interrupción pendiente correspondientes dentro del registro Cause. Es decir, el hardware deja constancia de que existe una solicitud de atención pendiente.

Sin embargo, el procesador solo acepta y atiende una interrupción cuando se cumplen dos condiciones simultáneas en el registro Status: el bit global IE (bit 0) debe estar activado y el bit de máscara correspondiente al nivel de interrupción del dispositivo (bit 8, IM0 en MARS) también debe estar activado. Si cualquiera de estos dos bits permanece en cero, el procesador ignora por completo las señales que llegan. No se produce la transferencia de control hacia la dirección 0x80000180, no se actualiza el registro EPC con la dirección de retorno, no se modifica el bit EXL del registro Status y no se ejecuta ninguna instrucción de la rutina de servicio. El flujo del programa principal continúa sin ninguna alteración aparente.

Las consecuencias son significativas, debido a que aunque los dispositivos están generando interrupciones de forma correcta, el programa no se entera de ellas, si el código principal se encuentra en un bucle de espera que consulta una variable actualizada por la rutina de interrupción (como ocurre con la variable pulsadorActivado en el ejercicio del semáforo), esa variable nunca cambiará de valor, porque la rutina de servicio nunca llega a ejecutarse. El programa permanecerá bloqueado indefinidamente en la espera, a pesar de que el usuario haya pulsado la tecla correcta.

De manera análoga, en el ejercicio del buffer circular de teclado, los caracteres tecleados no se almacenarían, puesto que la interrupción encargada de leerlos y filtrarlos no se atendería.

Además, el hecho de que las interrupciones queden pendientes en el registro Cause puede generar efectos secundarios si más adelante se habilitan las interrupciones en el procesador. En ese momento, todas las solicitudes acumuladas podrían atenderse de golpe o en un orden

no previsto, lo que complicaría el comportamiento del sistema. Por esta razón, la práctica recomendada es habilitar primero las interrupciones en el procesador y después en los dispositivos, o realizar ambas habilitaciones de forma coordinada antes de entrar en los bucles principales de espera.

El sistema pierde completamente la capacidad de respuesta asíncrona y se comporta, a efectos prácticos, como si las interrupciones no existieran. Únicamente cuando la habilitación se completa también en el registro Status del procesador, las señales generadas por el teclado y la pantalla consiguen detener el programa principal, transferir el control a la rutina de servicio y permitir un tratamiento eficiente de los eventos de entrada/salida.

9. Procesamiento de interrupciones.

El procesamiento de una interrupción de reloj en un sistema MIPS32 constituye un ejemplo claro de cómo el hardware y el software colaboran para atender un evento asíncrono de forma ordenada y transparente. Se va a describir de manera detallada y secuencial todo lo que ocurre desde el instante en que se genera el evento hasta que la rutina de servicio finaliza y el programa principal recupera el control.

a) Describe paso a paso qué ocurre cuando se produce una interrupción de reloj:

El proceso comienza cuando el contador interno del procesador (registro Count del Coprocesador 0) alcanza el valor previamente programado en el registro Compare. En ese momento el hardware del procesador activa el bit de interrupción pendiente correspondiente al temporizador dentro del registro Cause. Si las interrupciones están habilitadas de forma global (bit IE del registro Status) y la máscara del nivel de interrupción del reloj también está activa, el procesador reconoce la solicitud y decide aceptar la interrupción.

A partir de ese instante el hardware realiza una serie de acciones automáticas e indivisibles. En primer lugar, captura el valor actual del contador de programa y lo almacena en el registro EPC, de modo que más tarde se sepa exactamente a qué dirección se debe regresar.

A continuación modifica el registro Status: activa el bit EXL (Exception Level) para indicar que el procesador se encuentra en modo excepción y, al mismo tiempo, deshabilita temporalmente la aceptación de nuevas interrupciones. Por último, fuerza el contador de programa con la dirección fija del vector de excepciones generales, que en MARS es 0x80000180. Con estas operaciones el hardware garantiza que el control se transfiera de manera segura a la rutina de servicio y que el estado crítico del procesador quede preservado.

Una vez que la ejecución llega a la dirección 0x80000180, comienza la intervención del software, es decir, de la rutina de servicio de interrupción, lo primero que debe hacer esta rutina es guardar el contexto de los registros de propósito general que va a utilizar.

En la práctica se almacenan en una zona de memoria reservada (o en la pila) aquellos registros que la propia rutina modificará, típicamente \$at, \$v0, \$v1, \$a0-\$a3, \$t0-\$t9 y cualquier otro que se emplee. Este salvado es responsabilidad exclusiva del software; el hardware no lo realiza de forma automática.

Después de preservar el contexto, la rutina lee el registro Cause para confirmar que la interrupción pendiente corresponde realmente al reloj y no a otra fuente. Una vez identificada la causa, el software ejecuta las acciones propias del servicio: actualizar contadores de tiempo, cambiar el estado de una variable (por ejemplo, el color del semáforo), reprogramar el siguiente valor del registro Compare para generar la próxima interrupción de reloj, o cualquier otra tarea que el programa necesite realizar de forma periódica. Estas operaciones constituyen el núcleo del tratamiento y son completamente definidas por el programador.

Cuando el servicio ha terminado, la rutina de software restaura uno a uno los registros de propósito general que había guardado al principio, devolviéndolos exactamente a los valores que tenían antes de la interrupción. Finalmente ejecuta la instrucción `eret` (Exception Return); esta instrucción es interpretada por el hardware, que realiza las últimas acciones automáticas: restaura el bit EXL del registro Status (volviendo a habilitar las interrupciones si correspondía) y carga en el contador de programa el valor que estaba guardado en EPC. De este modo el procesador reanuda la ejecución del programa principal exactamente en el punto en que fue interrumpido.

Resumiendo, el hardware se encarga de detectar el evento, salvar la dirección de retorno en EPC, modificar el registro Status, deshabilitar interrupciones anidadas y redirigir el flujo hacia el vector de excepciones. El software, por su parte, se ocupa de guardar y restaurar los registros de propósito general, identificar la causa concreta de la interrupción, ejecutar las acciones de servicio y devolver el control mediante `eret`. Esta división clara de responsabilidades permite que una interrupción de reloj se atienda de forma eficiente, segura y transparente para el programa que estaba en ejecución.

b) ¿Por qué es importante guardar el contexto (registros generales, EPC, Status) al entrar en la rutina?

La preservación del contexto al entrar en una rutina de servicio de interrupción constituye una condición indispensable para que el mecanismo de interrupciones funcione de manera correcta y transparente en un sistema MIPS32. El contexto comprende tanto los registros de propósito general como los registros especializados EPC y Status del Coprocesador 0. Cada uno de estos elementos cumple una función distinta y su pérdida o alteración indebida produce efectos graves sobre la ejecución del programa interrumpido.

Los registros de propósito general contienen los valores temporales, direcciones, contadores y resultados intermedios que el programa principal estaba utilizando en el preciso instante en que se produjo la interrupción. La rutina de servicio necesita emplear algunos de estos registros para realizar su trabajo: leer el registro Cause, actualizar variables, reprogramar el temporizador o manipular datos.

Si la rutina modifica cualquiera de esos registros sin haber guardado previamente su valor original, al regresar al programa principal este encontrará datos corruptos, por ejemplo: Un contador que debería seguir incrementándose podría aparecer con un valor distinto, un puntero podría apuntar a una dirección incorrecta o un cálculo a medias podría continuar con operandos erróneos. La única forma de evitar esta corrupción es que el software de la rutina de servicio salve, antes de cualquier modificación, todos los registros que vaya a utilizar y los restaure de forma exacta antes de ejecutar la instrucción `eret`.

El registro EPC almacena la dirección de memoria a la que debe retornar el procesador una vez finalizado el tratamiento de la interrupción, aunque el hardware se encarga automáticamente de escribir en EPC la dirección correcta en el momento en que acepta la interrupción, este valor forma parte esencial del contexto que debe protegerse. Si durante la ejecución de la rutina de servicio se produjera una segunda excepción o si el software modificara accidentalmente el contenido de EPC, el retorno se realizaría a una dirección incorrecta. El programa principal podría reanudar la ejecución en un punto distinto del esperado, provocando saltos indebidos, bucles infinitos o la ejecución de instrucciones que no corresponden al flujo original. Por tanto,

aunque el hardware realiza la escritura inicial de EPC, el sistema en su conjunto depende de que este valor permanezca intacto hasta que la instrucción *eret* lo utilice.

El registro Status contiene el estado de habilitación de interrupciones, el bit EXL (Exception Level) y otras banderas de control del procesador, cuando se acepta una interrupción, el hardware activa automáticamente el bit EXL y deshabilita la aceptación de nuevas interrupciones para evitar anidamientos no controlados. Si la rutina de servicio necesita modificar temporalmente otros bits de Status (por ejemplo, para habilitar interrupciones de mayor prioridad o para cambiar el modo de operación), debe guardar previamente el valor original.

Al finalizar, es necesario restaurar Status de forma coherente para que el procesador recupere exactamente el mismo estado de habilitación que tenía antes de la interrupción, ya que una restauración incorrecta de Status podría dejar las interrupciones permanentemente deshabilitadas, habilitar fuentes que debían permanecer enmascaradas o mantener el procesador en modo excepción, impidiendo el correcto funcionamiento posterior del sistema.

La importancia de guardar el contexto completo se hace especialmente visible en programas que combinan varias fuentes de interrupción o que mantienen variables de estado críticas, como ocurre en las prácticas del buffer circular de teclado y del semáforo controlado por pulsador. En estos casos, el programa principal puede encontrarse en medio de una espera, de una actualización de mensajes o de una consulta a variables compartidas. Cualquier alteración no restaurada de los registros generales, de EPC o de Status rompería la lógica del programa, produciendo comportamientos impredecibles, pérdidas de caracteres, fallos en los temporizadores o bloqueos indefinidos.

10. Interrupciones de reloj y control de ejecución.

Las interrupciones de reloj constituyen una herramienta fundamental para ejercer control temporal sobre la ejecución de los programas en un sistema MIPS32. Gracias a ellas, el procesador puede recuperar periódicamente el control del sistema aunque el programa en ejecución se encuentre en una situación anómala o haya superado los límites de tiempo establecidos.

a) Explica cómo una interrupción de reloj puede usarse para:

- Evitar que un programa quede en un bucle infinito.

Una de las aplicaciones más importantes de la interrupción de reloj es evitar que un programa quede atrapado de forma indefinida en un bucle infinito. Cuando un programa contiene un error lógico que provoca un ciclo sin salida (por ejemplo, una condición de terminación que nunca se cumple), el procesador permanecería ejecutando ese bucle para siempre si no existiera un mecanismo externo de recuperación.

Al programar una interrupción de reloj periódica, el hardware genera de manera automática una solicitud de atención cada cierto intervalo de tiempo, independientemente de lo que esté haciendo el programa principal, cuando se acepta la interrupción, el control se transfiere a la rutina de servicio. Esta rutina puede incrementar un contador de “tiempo de ejecución”, comprobar si el programa actual lleva demasiado tiempo sin progresar o simplemente forzar un cambio de contexto.

De este modo, aunque el programa se encuentre en un bucle infinito, la interrupción de reloj garantiza que el sistema operativo o el monitor de ejecución recuperen el control de forma periódica y puedan tomar medidas correctivas, como abortar el proceso o devolver el control al usuario.

- Finalizar programas que superan un tiempo máximo de ejecución.

La segunda aplicación directa es la finalización automática de programas que superan un tiempo máximo de ejecución previamente establecido. En muchos entornos, especialmente en

sistemas de evaluación automática, entornos educativos o sistemas de tiempo real, cada programa dispone de un límite temporal (por ejemplo, unos pocos segundos).

Para implementar este control se programa el registro Compare del Coprocesador 0 de manera que la interrupción de reloj se active exactamente cuando se alcanza el tiempo máximo permitido, mientras el programa se ejecuta con normalidad, el contador Count avanza. Si el programa termina antes de que Count llegue al valor de Compare, no ocurre ningún problema.

Sin embargo, si el programa sigue en ejecución cuando se produce la interrupción, la rutina de servicio interpreta ese evento como la superación del límite de tiempo, en ese momento el software puede realizar las acciones necesarias: mostrar un mensaje de error, liberar los recursos que el programa estuviera utilizando y terminar de forma controlada la ejecución del proceso. Así se evita que un programa defectuoso o excesivamente costoso en tiempo monopolice el procesador de manera indefinida.

En ambos casos la interrupción de reloj actúa como un mecanismo de supervisión independiente del flujo de instrucciones del programa que se está ejecutando. Su carácter asíncrono y periódico permite al sistema recuperar el control en situaciones en las que el propio programa ya no es capaz de hacerlo.

b) ¿Qué debe hacer el software si el programa finaliza antes de que ocurra la interrupción de reloj?

Cuando un programa termina su ejecución de forma normal antes de que se active la interrupción de reloj programada, el software tiene la responsabilidad de desactivar o cancelar de manera limpia ese temporizador pendiente. Si no se realiza esta acción, la interrupción se generará de todos modos cuando el registro Count alcance el valor almacenado en Compare, lo que puede provocar comportamientos indeseados una vez que el programa original ya ha finalizado.

Lo primero que debe hacer el software es impedir que la interrupción de reloj llegue a producirse. Existen varias formas de conseguirlo, sin embargo, la más habitual consiste en reprogramar el registro Compare con un valor muy lejano en el futuro o con un valor que el contador Count no pueda alcanzar en un tiempo razonable, de este modo se elimina la condición que

provocaría la interrupción.

Otra posibilidad es deshabilitar temporalmente la máscara de interrupción correspondiente al reloj en el registro Status (el bit IM adecuado) o incluso desactivar el bit global IE, aunque esta última opción debe hacerse con cuidado para no afectar a otras interrupciones que pudieran estar activas. En cualquier caso, la deshabilitación debe realizarse mientras las interrupciones están controladas, para evitar condiciones de carrera.

Además de cancelar la futura interrupción, el software debe asegurarse de que no quede ningún bit de interrupción pendiente asociado al reloj en el registro Cause. Si el temporizador hubiera llegado a activar el bit de pendiente justo en el momento de la finalización, es conveniente limpiarlo de forma explícita, de lo contrario, si más adelante se vuelven a habilitar las interrupciones, el procesador podría atender de inmediato una interrupción de reloj que ya no tiene sentido, ejecutando una rutina de servicio que ya no corresponde al contexto actual.

En el ámbito de las prácticas realizadas en MARS, esta limpieza resulta especialmente importante, debido a que los programas del buffer de teclado y del semáforo con pulsador utilizan esperas temporales y variables de estado compartidas. Si una interrupción de reloj programada para controlar un tiempo máximo se dispara después de que el programa haya terminado o haya cambiado de fase, la rutina de servicio podría modificar variables que ya no son válidas, alterar el estado del semáforo o interferir con una nueva ejecución del mismo programa. Por ello, antes de salir de forma ordenada, el código debe dejar el temporizador en un estado inerte.

Finalmente, una vez cancelada la interrupción de reloj, el software puede proceder a las tareas habituales de finalización: liberar recursos, restaurar el valor original del registro Status si se hubiera modificado, y devolver el control al sistema operativo o al entorno de ejecución de MARS de forma limpia. De esta manera se garantiza que el sistema queda en un estado coherente y preparado para ejecutar el siguiente programa sin interferencias residuales del temporizador anterior.

Concluyendo, cuando un programa finaliza antes de que ocurra la interrupción de reloj, el software debe desactivar de forma explícita el temporizador, limpiar cualquier condición de interrupción pendiente y dejar el procesador en un estado seguro. Solo así se evita que un evento programado para un contexto que ya no existe interfiera con la ejecución posterior del sistema.

11. Análisis y Discusión de los Resultados.

Para esta práctica se debe analizar el código y los registros del coprocesador 0 (Status, Cause, EPC), con el fin de verificar que interrupción en 0x80000180 identifica correctamente la fuente de la interrupción y preserva el contexto del programa interrumpido, que la lógica propia de cada ejercicio reacciona correctamente ante cada situación, y también hasta qué punto ambos programas logran realmente liberar al procesador de la espera activa o polling.

a. Mapa de memoria y el alcance real de la interrupción habilitada.

Al probar ambos programas se pudo confirmar que usan exactamente el mismo par de registros mapeados en memoria para el teclado, apoyándose en los syscalls estándar de MARS, que serían 4, 11 y 30 para imprimir y para medir el tiempo.

Elemento	Dirección / Segmento	Uso
Registro de control del teclado.	0xffff0000.	Se escribe el valor 2 una sola vez al inicio de main, habilitando el bit1 (interrupción del teclado) de forma permanente.
Registro de datos del teclado.	0xffff0004.	Leído únicamente dentro de interrupción, nunca desde el programa principal.
Pantalla mapeada en memoria.	No utilizada.	La salida se hace con syscall 4 (cadena) y syscall 11 (carácter), no hay E/S de pantalla por interrupción.
tecladoBuffer, punteroEscritura, contadorCaracteres, pulsadorActivado, tiempoFin.	.data, a partir de 0x10010000.	Reservadas en el orden declarado.
guardaAT, guardaV0, guardaA0, guardaT0, guardaT1 (y guardaT2 en el buffer).	.kdata, a partir de 0x90000000.	Completamente separadas del segmento de usuario, sin ningún riesgo de choque con las variables del programa principal.

En ambos programas el único dispositivo verdaderamente gobernado por una interrupción de hardware es el teclado. El temporizador es llevado a cabo por software con el syscall 30 y la pantalla se maneja como normalmente se hace.

b. Ciclo de interrupción, registros Status/Cause y guardado de contexto.

Se verificó instrucción por instrucción que ambos programas habilitan las interrupciones exactamente de la misma manera al inicio de main, con las líneas siguientes.

```
mfc0 $t0, $12
```

```
ori $t0, $t0, 0x0101
```

```
mtc0 $t0, $12
```

El valor 0x0101 activa a la vez el bit0 (IE, habilitación global de interrupciones) y el bit8 (IM0, máscara del primer nivel de interrupción por hardware) del registro Status, sin alterar el resto de bits del registro gracias al uso de ori en lugar de una escritura directa. Se comprobó también que, para deshabilitar temporalmente las interrupciones, ambos programas usan la máscara 0xfffffe, que es el complemento a uno de 0x0101. Al aplicarla con and se apagan únicamente esos dos bits, dejando intacto cualquier otro bit del Status que MARS pudiera estar usando internamente, deshabilitando así interrupciones sin afectar al resto del registro.

Etapas	Instrucciones clave	Comportamiento verificado
Habilitación	ori \$t0, \$t0, 0x0101 sobre Status.	Activa IE (bit0) e IM0 (bit8).
Ocurrencia del evento	Tecla pulsada (pulsadorActivado).	El procesador transfiere el control a 0x80000180.
Identificación de la fuente	mfc0 \$k0, \$13 y andi \$k0, \$k0, 0x0100.	Distingue la interrupción de teclado (bit8 de Cause) de cualquier otra causa. Si el bit es 0, salta directo a salirInterrupcion.
Guardado de contexto	sw de \$at, \$v0, \$a0, \$t0, \$t1 (y \$t2 en el buffer) sobre .kdata.	Preserva los registros que se pueden modificar directa o indirectamente.
Restauración	lw en orden inverso, seguido de eret.	El programa interrumpido continúa sin ninguna alteración.

En cuanto al guardado de contexto. \$a0, \$t0 y \$t1 (y \$t2 en el buffer) se sobrescriben dentro de la rutina, por lo que su respaldo es obligatorio. \$at por otro lado, no está en ninguna instrucción de la parte de interrupción, pero sí se preserva. Las instrucciones la y li con direcciones o

inmediatos de 32 bits, se expanden internamente a lui/ori que utiliza \$at como registro intermedio, de modo que aunque en el algoritmo nunca se escribe, el ensamblador sí lo hace por debajo, y de no respaldarlo se correría el riesgo de corromper el \$at del programa interrumpido.

Por su parte, \$v0 se guarda y se restaura en ambos manejadores sin que ninguna instrucción de su cuerpo lo modifique realmente, ya que ninguna de las dos rutinas ejecuta ningún syscall. Su respaldo no es totalmente necesario tal como quedó escrito el código, y es más una práctica defensiva, importante sobre todo si en el futuro se amplía la rutina con alguna llamada al sistema. Tampoco se guarda \$ra porque ninguna de las dos rutinas ejecuta jal.

c. Análisis del Buffer de Teclado.

El filtro implementado en interrupcion se apoya en dos comparaciones sin signo consecutivas. Se constató que estas comparaciones delimitan el rango de 'A' a 'Z', sin errores de límite. El código 65 (A) pasa el primer filtro por no ser menor que 65, y el código 90 (Z) pasa el segundo por ser menor que 91, mientras que 97 (a, minúscula), cualquier número o signo de puntuación quedan descartados antes de tocar el buffer. El buffer tecladoBuffer reserva 100 bytes y punteroEscritura avanza con cada carácter válido. Cuando ese puntero llega a 100, la propia rutina lo reinicia a 0 (li \$t1, 0 antes de guardarPuntero), convirtiendo así al arreglo en un buffer circular. No obstante, contadorCaracteres que es la variable que usa imprimirBuffer para decidir cuántas posiciones recorrer, se queda en 100 una vez alcanzado ese valor (beq \$t1, \$t2, salirInterrupcion con \$t2 = 100) y ya no vuelve a incrementarse. Tras experimentar con distintos casos, se obtuvo:

Caso	Comportamiento
0 caracteres en 20 segundos.	contadorCaracteres permanece en 0, bucleImprimir termina en su primera comparación, se imprime únicamente el encabezado.
Entre 1 y 100 caracteres válidos.	El orden impreso coincide exactamente con el orden en que se tecló cada letra.
Más de 100 caracteres válidos.	El puntero comienza a sobrescribir desde la posición 0 en adelante, pero imprimirBuffer siempre recorre las posiciones 0 a 99 en orden de índice, no en orden de llegada. El resultado es una mezcla de la segunda vuelta de caracteres (al inicio del arreglo) con la cola de la primera vuelta que aún no fue sobrescrita (al final del arreglo), de forma que lo impreso deja de seguir el original desde la pulsación número 101.
Tecla mantenida presionada.	Se genera una interrupción por cada repetición, cada una tratada como una pulsación independiente.
Comparación de tiempo cerca del límite de 20 segundos.	El uso de sltu para comparar el tiempo actual contra tiempoFin maneja correctamente el caso en que el contador de milisegundos del sistema se acerca a su desbordamiento, gracias a la aritmética sin signo.

Lo más importante que podemos notar es que mientras el ciclo de espera sigue consultando el reloj en espera de los 20 segundos, las interrupciones del teclado permanecen habilitadas durante todo ese lapso de tiempo. La captura de caracteres y la medición del tiempo ocurren sin que una tarea bloquee a la otra, siendo que las interrupciones solo se deshabilitan muy brevemente entre el fin del temporizador y el reinicio del buffer para evitar que una tecla pulsada a mitad de la impresión modifique punteroEscritura o contadorCaracteres mientras esos valores están siendo leídos y reiniciados.

d. Análisis del Semáforo con Pulsador.

El sistema no reacciona al botón de forma inmediata en el programa principal, sino que utiliza una variable intermedia para comunicarse, cuando se presiona la letra `s` la rutina de interrupción detecta la coincidencia y cambia el valor de la variable `pulsadorActivado` a 1, mientras que el programa principal (`cicloPrincipal`) se queda atrapado en un bucle (`esperaPulsador`) consultando constantemente dicha variable mediante las instrucciones `lw` y `beq` hasta comprobar que dejó de ser cero y poder continuar.

Esto significa que, aunque la captura del carácter este dada por una interrupción real de hardware, la sincronización entre esa interrupción y el hilo principal se lleva a cabo mediante una bandera consultada por sondeo. Mientras el semáforo permanece en verde esperando al usuario, el procesador no queda libre, sino que se mantiene ejecutando el mismo par de instrucciones sobre `pulsadorActivado`.

Una vez detectada la pulsación, el programa deshabilita `Status` con la misma máscara `0xfffffefe`, y esa deshabilitación se mantiene durante toda la fase de amarillo que es de 10 segundos y la de rojo de 30 segundos, de manera que durante 40 segundos completos las interrupciones del teclado permanecen apagadas a nivel de procesador, y solo se reactivan al final de `esperaVerde`, justo antes de reiniciar el ciclo. Esto logra el efecto deseado de que una pulsación de la tecla `s` durante el amarillo o el rojo no altere la secuencia en curso, pero conviene señalar que es porque con `IE` en 0, el procesador simplemente no atiende ninguna interrupción, sin importar cuál sea. Si el bit de listo no se limpia mientras la interrupción está enmascarada, una pulsación ocurrida durante el amarillo o el rojo podría quedar pendiente y ser atendida de inmediato en cuanto `Status` vuelve a habilitarse al entrar de nuevo en verde, en lugar de perderse por completo.

En esa situación la letra `s` presionada, por ejemplo, durante la fase roja no se pierde sino que se posterga, y podría llegar a un cambio casi inmediato hacia el amarillo apenas comienza el nuevo ciclo verde, sin darle tiempo al usuario de ver el mensaje de esperando pulsador.

En la secuencia se verificó que las duraciones y los mensajes coinciden exactamente con lo pedido, con 20 segundos entre la pulsación y el amarillo, 10 segundos entre el amarillo y el rojo, y 30 segundos entre el rojo y el verde, calculados siempre como `tiempoActual + duración`

inmediatamente después de imprimir cada mensaje, y comparados con `sltu` en cada bucle de espera, con la misma técnica usada en el ejercicio del buffer.

e. Sondeo, interrupciones y el uso del procesador.

Los dos ejercicios dejan claro que usar interrupciones no elimina la espera activa, sino que la desplaza. En ninguno de los dos ejercicios el procesador llega a estar verdaderamente despejado en algún momento de su ejecución, porque en el buffer de teclado, el procesador pasa los 20 segundos en el bucle espera consultando el reloj del sistema una y otra vez, con la diferencia de que ese sondeo ya no es sobre el propio dispositivo de entrada sino sobre el temporizador, mientras el teclado se atiende por interrupción.

En el semáforo ocurre algo similar durante el amarillo y el rojo, pero además, durante el verde se suma un segundo tipo de sondeo, el de la bandera `pulsadorActivado`, que sí depende directamente del resultado de una interrupción pero que el programa principal sigue consultando de forma activa en vez de quedar suspendido a la espera de que la interrupción lo despierte. En la práctica anterior existía un único mecanismo, el sondeo directo del registro de estado del dispositivo, mientras aquí conviven tres mecanismos distintos dentro de los mismos dos programas, una interrupción de hardware para el teclado, un sondeo de software sobre el reloj del sistema para el temporizador, y un sondeo de software sobre una bandera en memoria para sincronizar la interrupción con el hilo principal.

f. Mecanismo de temporización utilizado.

Ninguno de los dos ejercicios implementa una interrupción de reloj real, al ambos reutilizar la misma secuencia basada en el `syscall 30`. Dichas instrucciones obtienen el tiempo actual del sistema en `$a0`, lo comparan con el valor previamente calculado como `tiempoActual + duración`, y repiten el ciclo mientras el tiempo actual siga siendo menor. El procesador es el que decide en cada vuelta del bucle si ya es momento de dejar de preguntar. Sin un dispositivo de temporización con interrupción propia, cualquier control de tiempo en MARS termina apoyándose en una forma de sondeo.

Después de todo lo expuesto, se puede afirmar que ambos programas cumplen con lo exigido, filtrando y almacenando únicamente letras mayúsculas en un buffer circular de 100 posiciones en el primer ejercicio, y ejecutando con precisión la secuencia temporizada del semáforo activada por la tecla `s` en el segundo, con un manejador de excepciones que identifica correctamente la fuente de la interrupción y guarda el contexto. El uso de interrupciones se aplica para la entrada por teclado, pero no elimina en su totalidad la espera activa, que se pasa al sondeo de un reloj de software y de una bandera de sincronización en memoria, lo que resalta la importancia de recurrir a un sistema operativo capaz de bloquear procesos y a temporizadores de hardware dedicados para lograr un procesador libre de sondeo.